

Web-based E-commerce Application with Integrated Product Recommendation System

Software Architecture Document (SAD)

CONTENT OWNER:

Huỳnh Quốc Sang

23521544@gm.uit.edu.vn

Nguyễn Thanh Bình

23521484@gm.uit.edu.vn

Dang Pham Nguyet Sang

23521336@gm.uit.edu.vn

MỤC LỤC

1	Documentation Roadmap.....	5
1.1	Purpose	5
1.2	Scope	5
1.3	Audience.....	5
1.4	SAD Organization	6
1.5	Stakeholder Concerns & Viewpoints	6
2	Architecture Background.....	7
2.1	Problem Background	7
2.1.1	Business Context & System Overview	7
2.1.2	Business Goals	7
2.1.3	Major Constraints.....	7
2.1.4	Key Functional Requirements.....	8
2.1.5	Quality Attribute Drivers	8
2.2	Solution Background.....	8
2.2.1	Architectural Approaches.....	8
2.2.2	Architecture Style Rationale (Lý do chọn mẫu thiết kế)	8
2.2.3	Alternative Approaches Considered & Trade-offs	9
3	Architectural Views.....	10
3.1	Context View.....	10
3.2	Container View.....	12
3.3	Component View	13
3.4	Module View	14
3.4.1	Layered Architecture & Dependency Graph	14

3.4.2	Coupling Analysis & Shared Kernel.....	14
3.5	Domain View.....	15
3.5.1	Entity Relationship Diagram (ERD)	15
3.5.2	Transactional Consistency & Invariants	17
3.6	Runtime View.....	17
3.6.1	OTP Login & Trusted Device Registration	17
3.6.2	Multi-Shop Checkout & Stock Reservation.....	18
3.6.3	Payment Callback (Webhook) & Asynchronous Fulfillment.....	20
3.6.4	Threading, Transaction & Async Boundaries.....	22
3.7	Deployment View.....	22
4	Data architecture.....	23
4.1	Hybrid Storage Strategy	23
4.2	Database Interaction Diagram	24
4.3	Caching & Locking Mechanisms	25
5	Api architecture	25
5.1	REST Conventions & Error Handling.....	25
5.2	API Catalog	26
6	Security architecture.....	29
6.1	Authentication Sequence Diagram	29
6.2	Role-Based Access Control (RBAC) Matrix	30
6.3	Security Boundary & Mitigations	31
7	Quality attribute analysis.....	31
7.1	Performance	31
7.2	Scalability	32

7.3	Observability & Auditability	32
8	Devops & operations	34
8.1	DevOps Pipeline Diagram	34
8.2	Logging, Monitoring & Alerting Stack	34
9	Risks & technical debt.....	35
10	Architecture decision records (adr)	36
11	Requirement traceability	38

1 Documentation Roadmap

1.1 Purpose

Tài liệu Kiến trúc Phần mềm (Software Architecture Document - SAD) này cung cấp một cái nhìn tổng quan toàn diện và sâu sắc về cấu trúc kỹ thuật của hệ thống Backend ShopMart. Tài liệu ghi lại các quyết định thiết kế kiến trúc quan trọng, các mẫu kiến trúc (architectural patterns) được áp dụng, cấu trúc phân rã module, cơ chế bảo mật, thiết kế luồng dữ liệu, và chiến lược vận hành nhằm đáp ứng các mục tiêu kinh doanh và thuộc tính chất lượng (Quality Attributes) của một nền tảng thương mại điện tử đa người bán (multi-seller).

1.2 Scope

Tài liệu này bao phủ toàn bộ hệ thống lõi Backend của dự án ShopMart:

- **Ứng dụng chính:** Spring Boot 3.4.1 (Java 21), RESTful API và WebSocket server.
- **Cơ sở dữ liệu:** Hệ thống lưu trữ lai (Hybrid Storage) gồm MySQL (RDBMS cho transaction), MongoDB (Document Store cho logs/chats) và Redis (In-memory Cache/Session Store).
- **Tích hợp ngoại vi:** Cổng thanh toán (MoMo, VNPAY), Dịch vụ lưu trữ đám mây (Cloudinary, GCS), Trợ lý ảo AI (Groq API với LLM Llama-3.3), và Hệ thống gửi Email.
- **Môi trường triển khai:** Kiến trúc Container hóa chạy trên nền tảng Docker/Kubernetes.

1.3 Audience

- Enterprise Architects & Lead Engineers: Đánh giá độ trưởng thành kỹ thuật, quản lý nợ kỹ thuật và định hình lộ trình phát triển.
- Software Developers (Backend/Frontend): Định hướng triển khai code theo đúng quy chuẩn kiến trúc (architectural boundaries) và tích hợp API.

- QA / Tester: Thiết kế các ca kiểm thử tích hợp (integration tests) và kiểm thử hiệu năng (load tests) dựa trên luồng xử lý runtime.
- DevOps / SRE Engineers: Cấu hình hạ tầng triển khai, thiết lập hệ thống giám sát (monitoring), và xây dựng kịch bản tự động scale.

1.4 SAD Organization

Tài liệu được tổ chức theo tiêu chuẩn IEEE 1471 và mô hình biểu diễn kiến trúc C4 (Context, Container, Component, Code/Module).

1.5 Stakeholder Concerns & Viewpoints

Bảng dưới đây ánh xạ các mối quan tâm của các bên liên quan (Stakeholders) vào các góc nhìn kiến trúc (Viewpoints) tương ứng:

Stakeholder	Concerns (Mối quan tâm)	Architectural Viewpoint
Business Owner / PM	Thời gian ra mắt tính năng mới, chi phí vận hành hạ tầng, tích hợp các đối tác thanh toán.	Context View, Solution Background
Lead Developer	Code có bị coupling không? Khả năng viết test độc lập, quy tắc phân lớp dữ liệu.	Component View, Module View, Domain View
Security Officer	Chống rò rỉ token, phòng ngừa replay attack, bảo mật callback IPN, phân quyền đa vai trò.	Security Architecture, Runtime View
DevOps / SRE	Khả năng chịu lỗi (resilience), chiến lược scale out, giám sát tài nguyên CPU/RAM, rollback khi lỗi.	Deployment View, DevOps & Operations
Database Administrator	Đảm bảo tính ACID, phòng ngừa overselling khi flash sale, hiệu năng query khi băng phình to.	Data Architecture, Domain View

2 Architecture Background

2.1 Problem Background

2.1.1 Business Context & System Overview

ShopMart là một sàn thương mại điện tử đa người bán (Multi-Seller E-Commerce Platform) kết nối ba nhóm đối tượng (Actors) chính: Buyer (Người mua hàng), Seller (Người bán hàng sở hữu các Shop riêng), và Admin (Quản trị viên sàn). Hệ thống hoạt động theo mô hình giao dịch trực tuyến thời gian thực, nơi người mua có thể chọn hàng từ nhiều shop khác nhau đưa vào một giỏ hàng và tiến hành checkout trong một giao dịch duy nhất.

2.1.2 Business Goals

- Multi-Seller Fulfillment: Cho phép xử lý đơn hàng độc lập theo từng shop từ một giao dịch checkout tổng.
- Real-time Interaction: Giao tiếp tức thời giữa Buyer và Seller qua chat và thông báo in-app.
- AI-powered Experience: Hỗ trợ Buyer tìm kiếm, so sánh sản phẩm và tư vấn mua hàng thông qua chatbot AI thông minh.
- Financial Trust: Thanh toán trực tuyến an toàn, tin cậy qua các ví điện tử và cổng thanh toán quốc gia (MoMo, VNPAY).

2.1.3 Major Constraints

- Công nghệ cốt lõi: Bắt buộc xây dựng trên hệ sinh thái Java / Spring Boot 3.x.
- Thời gian giao dịch: Luồng checkout và kiểm tra tồn kho phải hoàn tất dưới 2 giây để tránh drop-off rate.
- Hạ tầng cục bộ: Hỗ trợ chạy offline hoặc dev local dễ dàng mà không phụ thuộc hoàn toàn vào dịch vụ cloud trả phí (nhờ cơ chế Mock Payment và Local Storage).

2.1.4 Key Functional Requirements

- Xác thực: Đăng ký/Đăng nhập bằng Email/Password hoặc OTP qua SĐT; Đăng nhập không mật khẩu qua Thiết bị tin cậy (Trusted Device); Đăng nhập mạng xã hội (OAuth2 Google).
- Quản lý sản phẩm (Catalog): Quản lý danh mục đa cấp, thương hiệu, sản phẩm biến thể (SKU) và các thuộc tính động phục vụ cho bộ so sánh sản phẩm.
- Giao dịch: Giỏ hàng, Áp dụng Voucher/Coupon (hệ thống & shop), Checkout tạo đơn hàng, Thanh toán trực tuyến và đối soát Callback (IPN).
- Vận hành: Chat Buyer-Seller, Gửi thông báo In-app & Email, Quản lý đổi trả/hoàn tiền (Refund), Nhật ký sự kiện (Audit Log) cho Admin, Dashboard thống kê doanh thu cho Seller/Admin.

2.1.5 Quality Attribute Drivers

- Security (Bảo mật): Đảm bảo an toàn thông tin thanh toán, chống giả mạo request, và phân quyền chặt chẽ giữa các actor.
- Data Consistency (Tính nhất quán dữ liệu): Tuyệt đối không để xảy ra tình trạng bán vượt quá tồn kho thực tế (overselling) hoặc lệch tiền thanh toán.
- Scalability & Performance (Khả năng mở rộng và hiệu năng): Hệ thống phải đáp ứng lượng truy cập đọc lớn vào catalog sản phẩm và ghi đồng thời cao vào giỏ hàng/checkout trong các đợt flash sale.

2.2 Solution Background

2.2.1 Architectural Approaches

Hệ thống lựa chọn kiến trúc **Clean Architecture** làm mô hình chủ đạo. Toàn bộ logic nghiệp vụ cốt lõi được cô lập hoàn toàn vào lớp Domain, tách biệt khỏi các framework bên ngoài (như Spring, Hibernate) và các driver cơ sở dữ liệu.

2.2.2 Architecture Style Rationale (Lý do chọn mẫu thiết kế)

- Clean Architecture vs Traditional 3-Tier Layered: Trong kiến trúc 3 lớp truyền thống (Presentation → Business → Data Access), business logic bị phụ thuộc

chặt chẽ vào cấu trúc bảng cơ sở dữ liệu (Database Schema). Khi thay đổi database hoặc chuyển từ SQL sang NoSQL, toàn bộ code nghiệp vụ phải viết lại. Clean Architecture đảo ngược sự phụ thuộc (Dependency Inversion), hướng các module công nghệ phụ thuộc vào lõi nghiệp vụ. Điều này giúp ShopMart dễ dàng chuyển đổi nhà cung cấp lưu trữ hình ảnh (Cloudinary sang GCS) hoặc công thanh toán mà không ảnh hưởng đến logic đặt hàng.

- Hybrid Storage (SQL + NoSQL):
 - MySQL (ACID) được chọn cho Orders, Payments, Inventory và Users vì các nghiệp vụ này yêu cầu tính toàn vẹn dữ liệu tuyệt đối (Strong Consistency).
 - MongoDB (Document) được chọn cho Chat Messages và Audit Logs vì đây là dữ liệu có tần suất ghi cực cao, phi cấu trúc và kích thước tăng nhanh, cần khả năng scale out dễ dàng.
 - Redis (Cache/In-memory) được dùng để lưu các trạng thái ngắn hạn như OTP, JWT blacklist và cached products nhằm giảm tải cho RDBMS.

2.2.3 Alternative Approaches Considered & Trade-offs

- Option A: Pure Microservices Architecture
 - Xem xét: Chia nhỏ hệ thống thành các service độc lập: auth-service, order-service, catalog-service, chat-service.
 - Đánh giá trade-off: Microservices giúp scale độc lập từng phần nhưng làm tăng độ phức tạp vận hành lên gấp nhiều lần. Các vấn đề về Distributed Transaction (ví dụ: trừ kho ở inventory-service và tạo đơn ở order-service cần cơ chế Saga/2PC) và độ trễ mạng (network latency) giữa các service sẽ trở thành rào cản lớn cho một dự án đang phát triển.
 - Quyết định: Chọn Modular Monolith (Monolith phân rã module rõ ràng theo Clean Architecture). Khi hệ thống phình to, việc bóc tách các module này thành microservices rất đơn giản vì ranh giới nghiệp vụ (Bounded Contexts) đã được phân định rõ ở tầng Domain.
- Option B: Optimistic Locking cho Đặt hàng đồng thời

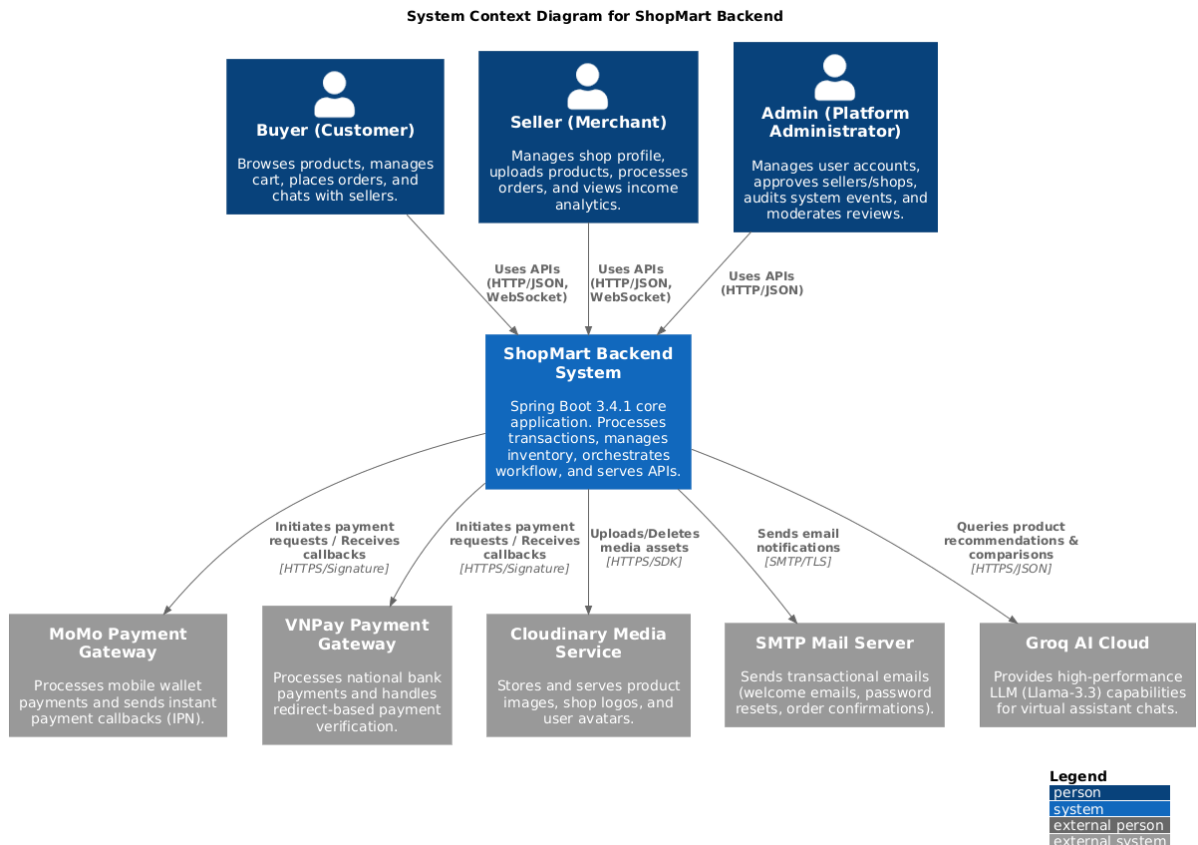
- Xem xét: Sử dụng annotation `@Version` của JPA để check xung đột khi cập nhật tồn kho SKU.
- Đánh giá trade-off: Optimistic lock hoạt động tốt khi xung đột thấp. Nhưng trong các đợt flash sale, khi hàng ngàn Buyer cùng tranh giành một SKU hot, việc liên tục rollback transaction do xung đột version sẽ tạo ra trải nghiệm người dùng rất tệ (user liên tục bị báo lỗi đặt hàng thất bại và phải thao tác lại).
- Quyết định: Chọn Pessimistic Locking (`SELECT ... FOR UPDATE`) kết hợp với cơ chế Stock Reservation (giữ chỗ tạm thời trong 15 phút). Dữ liệu kho được lock ở mức hàng trong thời gian cực ngắn chỉ để chuyển số lượng sang `reserved_stock`, giúp tối ưu hóa throughput ghi và nâng cao tỷ lệ checkout thành công.

3 Architectural Views

3.1 Context View

Góc nhìn Context xác định ranh giới của hệ thống ShopMart Backend, chỉ ra các tác nhân (Actors) tương tác trực tiếp và các hệ thống bên ngoài (External Systems) tích hợp với nó.

Hình: C4 System Context Diagram



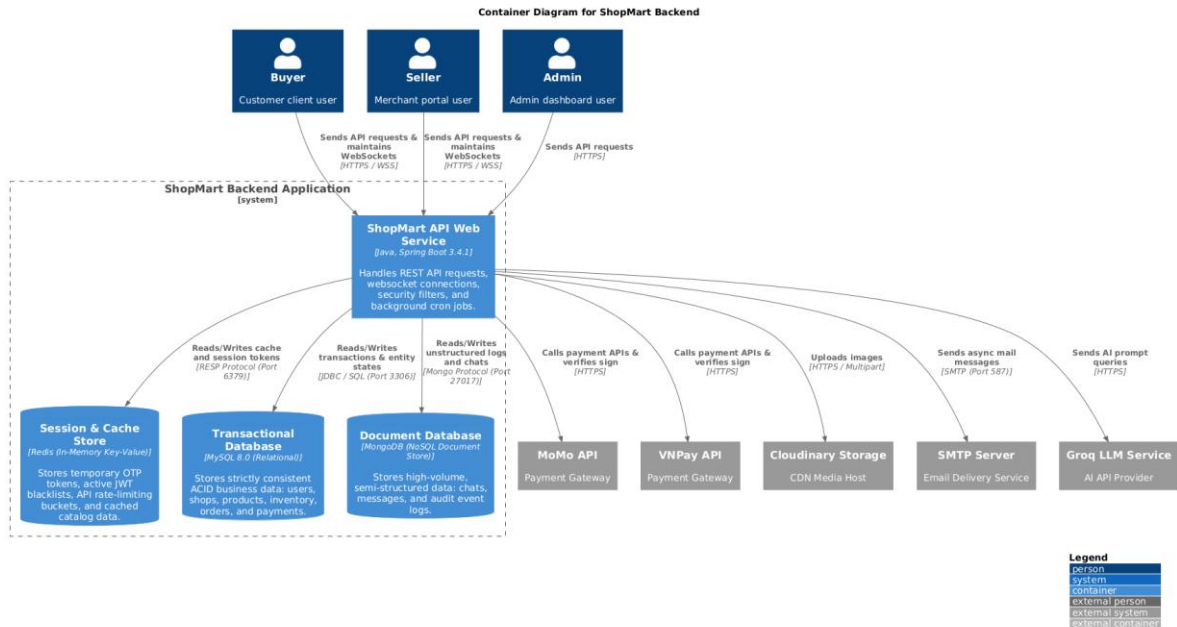
Hình: C4 System Context Diagram

Interaction Flow Description

- **Fulfillment Flow:** Buyer thực hiện đặt hàng trên Client app. ShopMart Backend tiếp nhận request, tiến hành giữ kho (Stock Reservation), tính toán voucher và tạo các đơn hàng tương ứng theo từng Shop.
- **Payment Flow:** Đối với đơn hàng thanh toán trực tuyến, ShopMart gửi yêu cầu tạo giao dịch sang MoMo/VNPay. Hệ thống nhận callback IPN trực tiếp từ cổng thanh toán qua kênh bảo mật để cập nhật trạng thái đơn hàng (PAID/CANCELLED).
- **AI Chatbot Flow:** Buyer gửi truy vấn tư vấn sản phẩm. Backend tiếp nhận, tổng hợp thông tin sản phẩm liên quan từ cơ sở dữ liệu để làm ngữ cảnh (Context Injection), gửi prompt tới Groq Cloud, và trả kết quả tư vấn tự nhiên về cho Buyer.

3.2 Container View

Góc nhìn Container phân rã hệ thống ShopMart thành các đơn vị thực thi độc lập (containers), mô tả công nghệ sử dụng, trách nhiệm và cách chúng giao tiếp với nhau.

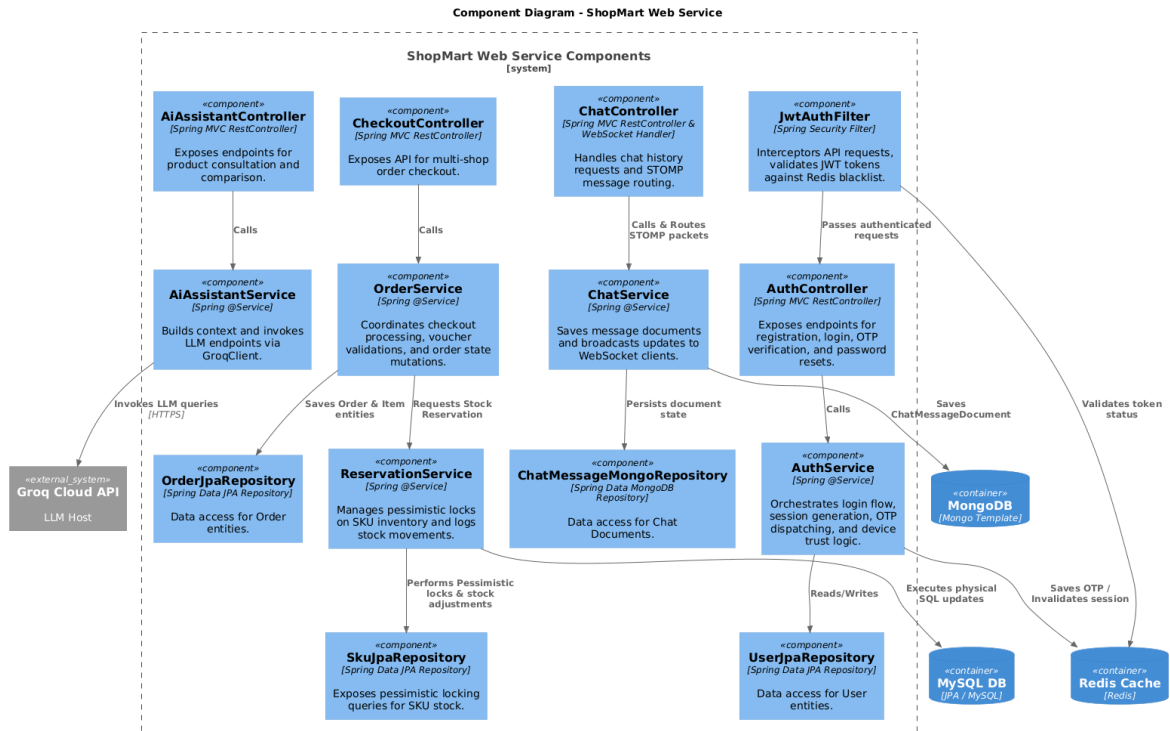


Container Responsibilities & Protocols

- **ShopMart API Web Service (Spring Boot):** Lõi xử lý chính. Giao tiếp với Client qua giao thức HTTPS (REST API) và WSS (WebSocket STOMP cho Chat). Giao tiếp với các database bằng driver chuyên dụng (JDBC cho MySQL, Mongo Driver cho MongoDB, Lettuce/Jedis cho Redis).
- **Redis Cluster:** Bộ nhớ đệm phân tán. Tăng tốc độ truy xuất dữ liệu đọc thông qua giao thức RESP. Quản lý vòng đời token, OTP và Rate limit.
- **MySQL Primary-Replica:** Hệ quản trị cơ sở dữ liệu quan hệ chính. Đảm bảo tính nhất quán dữ liệu ở mức cao nhất cho các transaction tài chính, thanh toán, kho hàng.
- **MongoDB Replica Set:** Lưu trữ tài liệu. Thiết kế tối ưu hóa cho ghi chép logs và tin nhắn chat real-time với schema linh hoạt.

3.3 Component View

Góc nhìn Component mô tả chi tiết cấu trúc bên trong của container lõi **ShopMart API Web Service**, chỉ ra các thành phần phần mềm chính và mối quan hệ phụ thuộc giữa chúng.



3.3.1 Component Dependency Explanation & Interface Contracts

- **JwtAuthFilter**: Filter chặn mọi request (ngoại trừ các endpoint public được cho phép tại SecurityConfig.java). Lớp này giải mã JWT bằng JwtService, kiểm tra blacklist trong Redis và nạp Principal (userId) vào Spring Security Context.
- **ReservationService**: Cung cấp các hợp đồng nghiệp vụ kho:
 - reserve(orderToken, skuId, qty): Khoá hàng tạm thời.
 - commit(orderToken): Khấu trừ tồn kho thực tế khi thanh toán thành công.
 - release(orderToken): Giải phóng hàng dự trữ nếu thanh toán thất bại.
 - restore(orderToken): Hoàn kho nếu đơn hàng bị hủy sau khi đã committed.

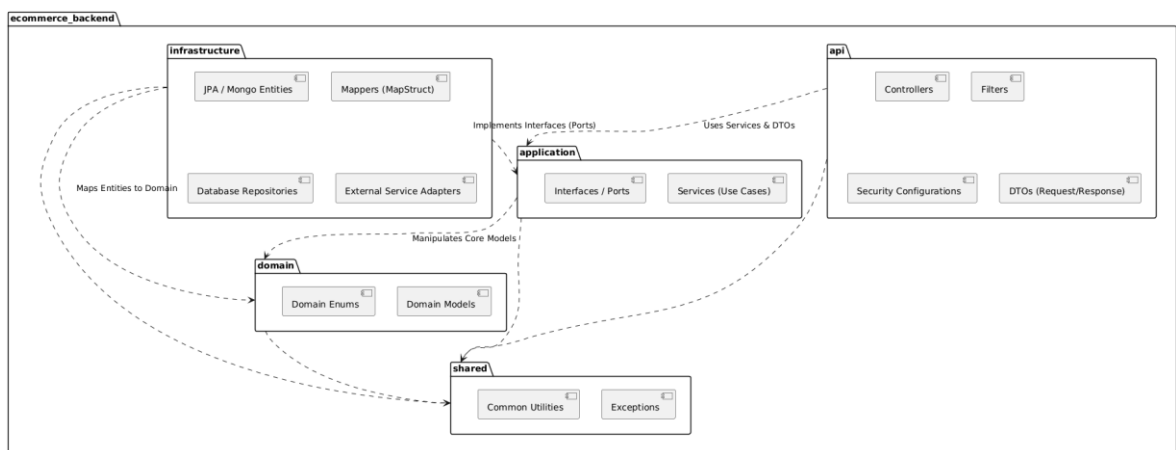
- **GroqClient (Component):** Lớp tích hợp hạ tầng giao tiếp HTTP với Groq AI Cloud, cấu hình sẵn timeout và API key bảo mật.

3.4 Module View

Góc nhìn Module mô tả cách phân chia mã nguồn trong IDE và chiều phụ thuộc của các gói code (packages).

3.4.1 Layered Architecture & Dependency Graph

ShopMart tuân thủ quy tắc phụ thuộc của Clean Architecture: **Sự phụ thuộc luôn hướng vào trong**. Các lớp bên ngoài biết về các lớp bên trong, nhưng các lớp bên trong tuyệt đối không được biết gì về các lớp bên ngoài.



3.4.2 Coupling Analysis & Shared Kernel

- **Domain Layer (Core):** Là Bounded Context trung tâm, chứa các quy tắc nghiệp vụ bất biến. Gói này hoàn toàn độc lập, không import bất kỳ class nào của Spring Framework hay Hibernate.
- **Shared Layer (Shared Kernel):** Chứa các util class và Exception cơ sở. Đây là gói duy nhất được phép import bởi tất cả các layer khác.
- **Low Coupling, High Cohesion:** Nhờ việc áp dụng các Interface tại tầng Application (như ReservationService, CloudStorageService) và triển khai (Implementation) tại tầng Infrastructure, mã nguồn đạt được độ loose coupling cao. Khi cần thay đổi thư viện map database hoặc chuyển đổi từ Cloudinary

sang Amazon S3, lập trình viên chỉ cần sửa code ở tầng Infrastructure mà không cần động chạm đến tầng API hay Business Service.

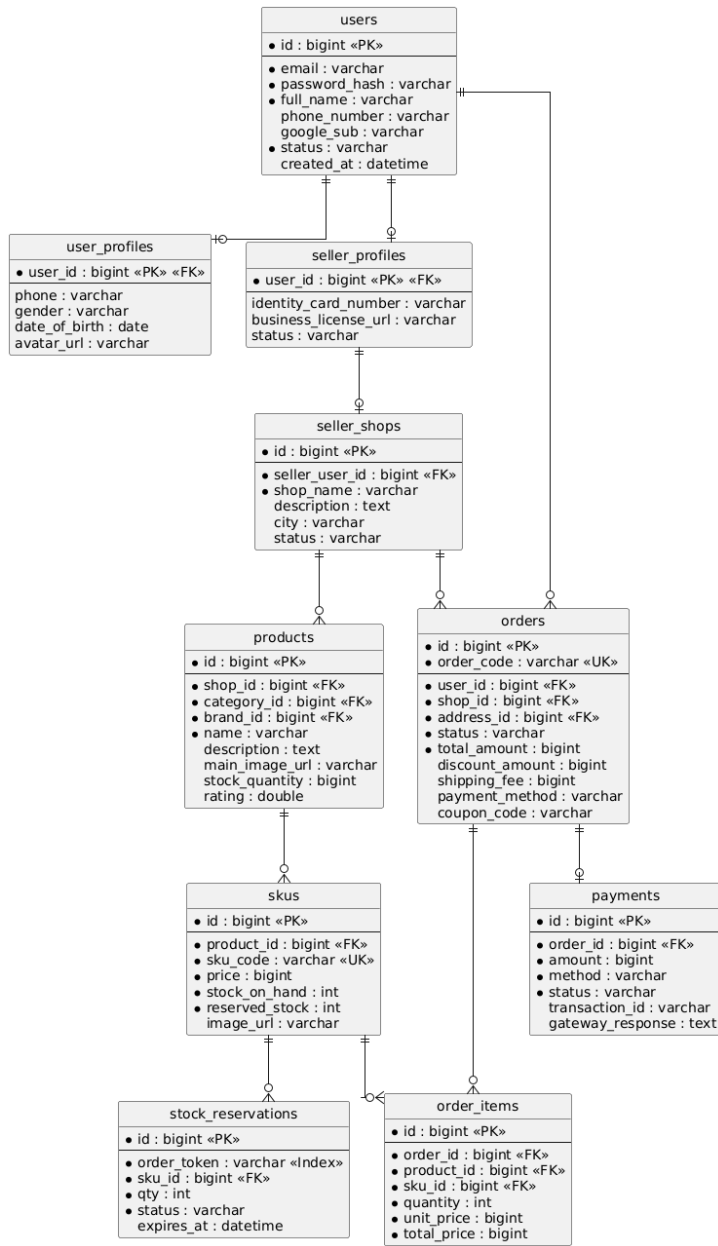
3.5 Domain View

Góc nhìn Domain tập trung mô tả mô hình dữ liệu nghiệp vụ của hệ thống dưới dạng các thực thể (Entities), nhóm thực thể (Aggregates), và mối quan hệ giữa chúng.

3.5.1 Entity Relationship Diagram (ERD)

Dưới đây là sơ đồ chi tiết các bảng quan hệ MySQL chính được ánh xạ từ cấu trúc database thực tế:

ShopMart Database ERD (MySQL Core)



3.5.2 Transactional Consistency & Invariants

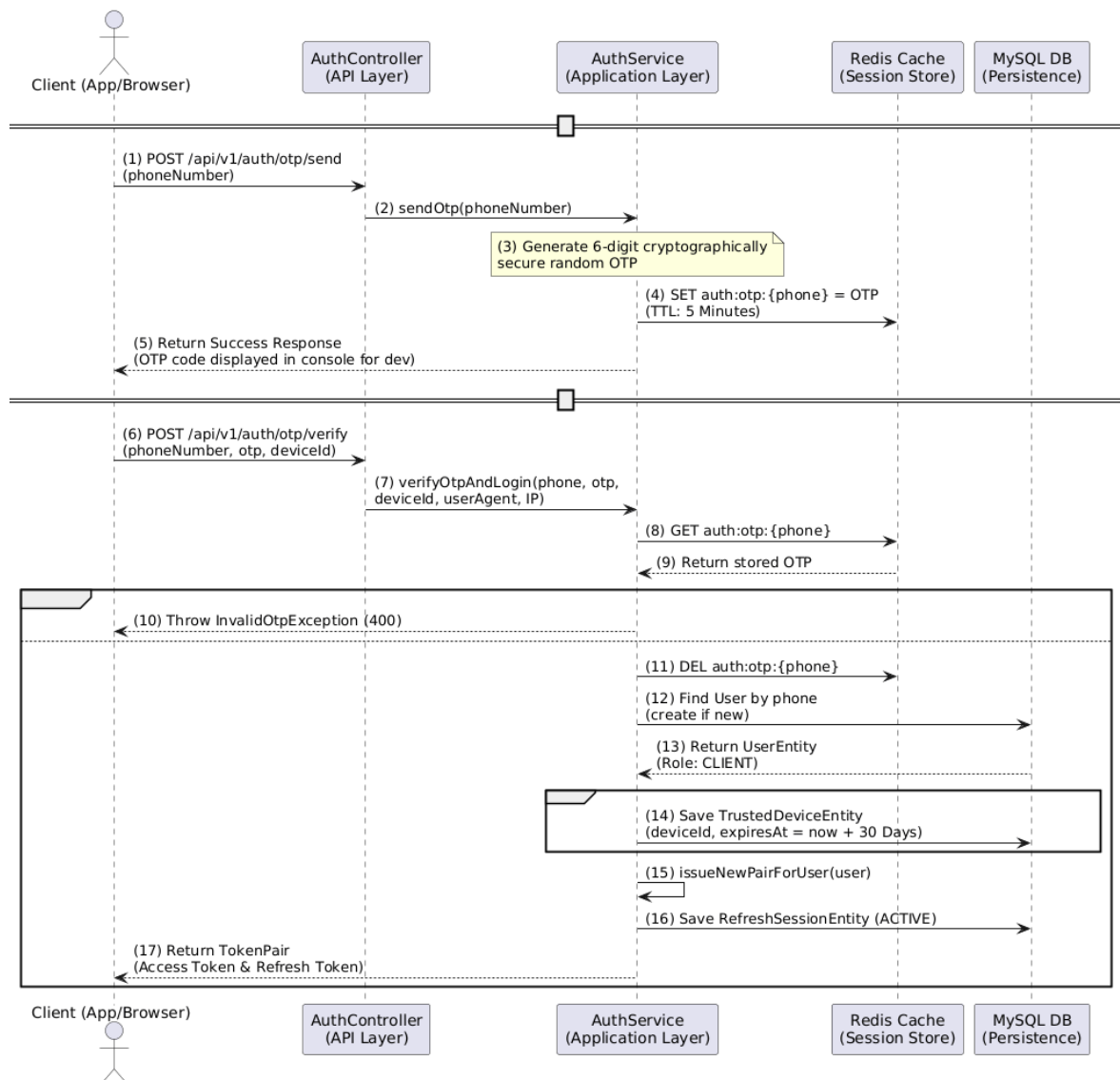
- **Invariants (Ràng buộc bất biến):**
 - Tồn kho khả dụng của một SKU luôn phải đảm bảo: $\text{AvailableStock} = \text{stock_on_hand} - \text{reserved_stock} \geq 0$. Ràng buộc này được thực thi nghiêm ngặt thông qua khóa bi quan (Pessimistic Lock) tại `ReservationServiceImpl.java`.
 - Trạng thái đơn hàng (`OrderStatus`) và trạng thái thanh toán (`PaymentStatus`) phải đồng bộ: Không thể chuyển đơn hàng sang trạng thái `PROCESSING` hoặc `PAID` nếu bản ghi payments tương ứng chưa đạt trạng thái `COMPLETED` (ngoại trừ thanh toán dạng COD).
- **Aggregate Boundaries:**
 - **Order Aggregate:** Gồm Order làm Aggregate Root và các OrderItem làm thực thể con. Mọi thay đổi trên Item (như thay đổi số lượng, đơn giá) chỉ được thực hiện thông qua root Order để kiểm soát tổng tiền và đảm bảo tính nhất quán dữ liệu.

3.6 Runtime View

Góc nhìn Runtime mô tả cách thức các thành phần trong hệ thống phối hợp hoạt động trong thời gian thực để thực hiện các luồng nghiệp vụ phức tạp.

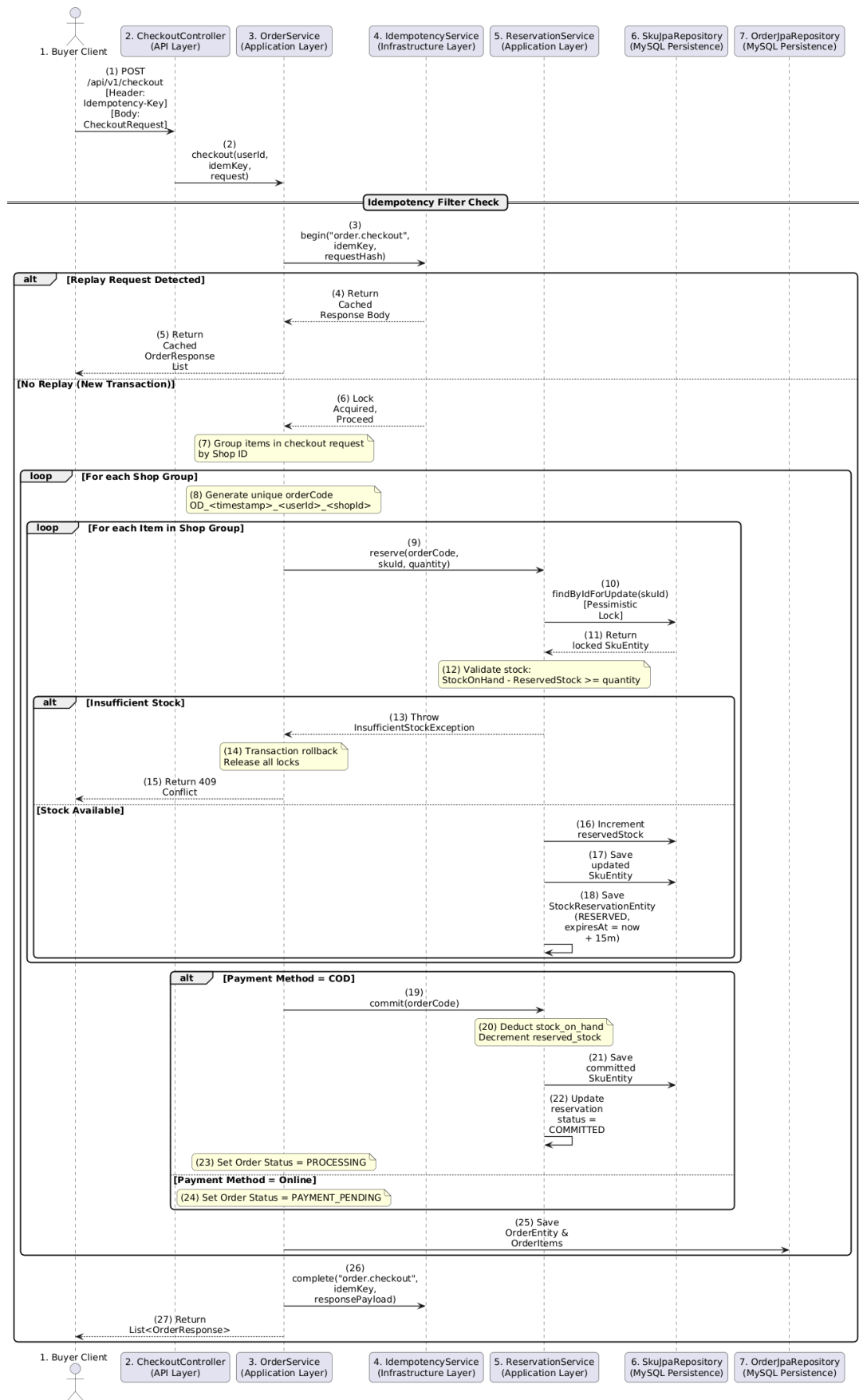
3.6.1 Runtime Flow Diagram: OTP Login & Trusted Device Registration

Quy trình xác thực an toàn qua OTP, đăng ký thiết bị tin cậy nhằm nâng cao trải nghiệm người dùng (bypass OTP cho các lần đăng nhập tiếp theo):



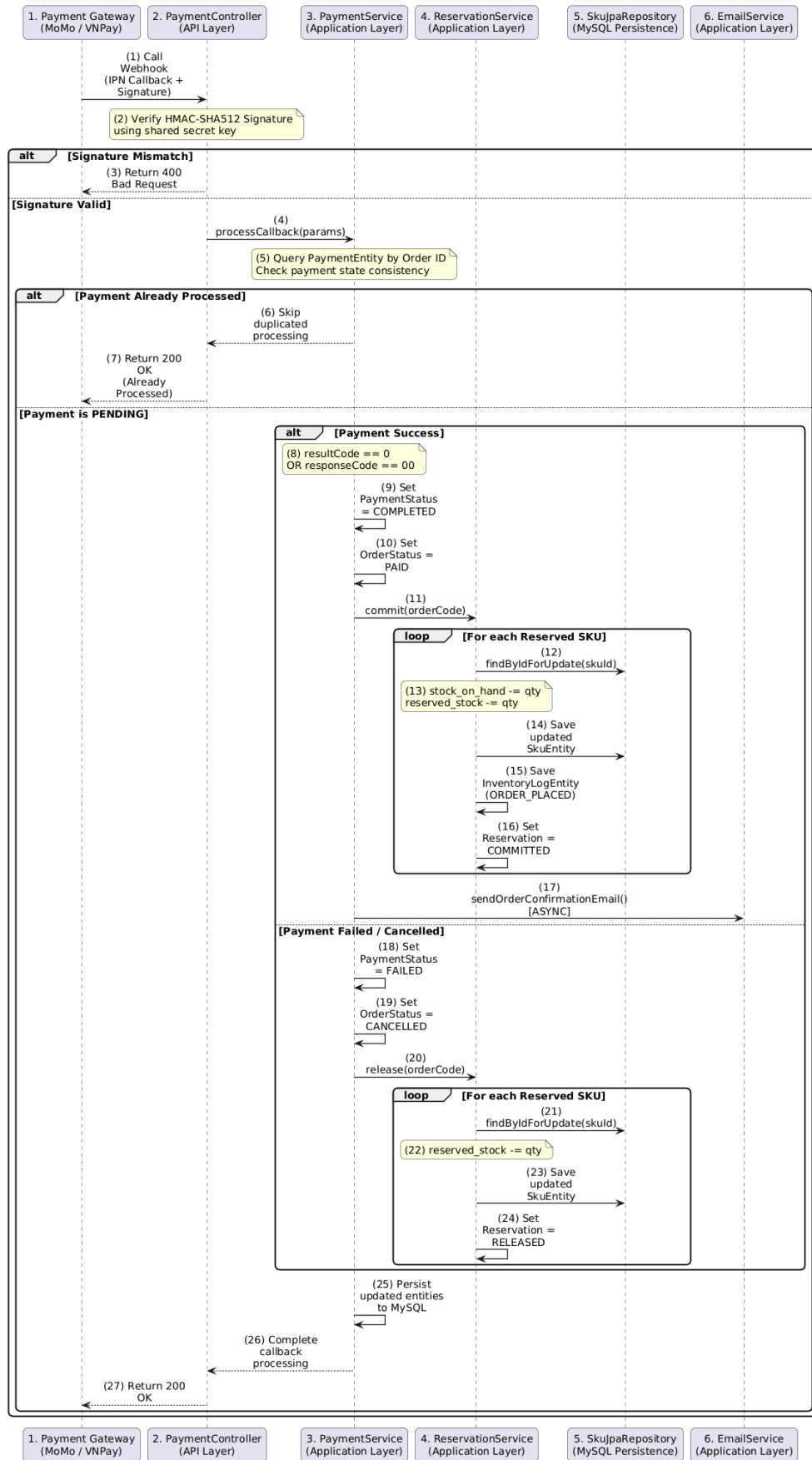
3.6.2 Runtime Flow Diagram: Multi-Shop Checkout & Stock Reservation

Mô tả chi tiết luồng xử lý Checkout đơn hàng chứa sản phẩm từ nhiều Shop khác nhau, kết hợp cơ chế kiểm soát kho hàng an toàn và phòng tránh trùng lặp đơn hàng nhờ Idempotency Key:



3.6.3 Runtime Flow Diagram: Payment Callback (Webhook) & Asynchronous Fulfillment

Quy trình tiếp nhận webhook thông báo thanh toán thành công/thất bại, đối soát chữ ký số an toàn và thực hiện xử lý nghiệp vụ kho sau khi giao dịch hoàn tất:



3.6.4 Threading, Transaction & Async Boundaries

- **Sync vs Async Boundaries:** Các luồng tương tác trực tiếp với khách hàng (như Checkout, Verify OTP, Payment Callback) được thực thi hoàn toàn đồng bộ (Synchronous) trên luồng HTTP worker (Tomcat Thread Pool) để trả về phản hồi tức thì. Các tác vụ phụ (gửi Email, ghi Event Log, phân tích hành vi) được chuyển giao sang luồng chạy nền (Background Thread Pool) thông qua annotation `@Async` hoặc đẩy vào hàng đợi tự động để giảm tải thời gian phản hồi API.
- **Scheduled Background Jobs:** Hệ thống sử dụng Spring `@Scheduled` (`ScheduledCleanupService.java`) để định kỳ thực hiện dọn dẹp dữ liệu rác chạy nền:
 - *Mỗi 60 giây:* Quét tìm và tự động giải phóng (release) các bản ghi `stock_reservations` đã quá hạn 15 phút mà chưa thanh toán để hoàn kho cho sản phẩm.
 - *2:00 AM hàng ngày:* Xóa các token reset mật khẩu hết hạn.
 - *3:00 AM hàng ngày:* Xóa các token refresh hết hạn hoặc đã bị thu hồi cách đó 30 ngày.

3.7 Deployment View

Góc nhìn Deployment mô tả cách thức đóng gói và phân bổ các container ứng dụng lên hạ tầng vật lý hoặc môi trường cloud Kubernetes thực tế.

Production Infrastructure Details

- **Load Balancer:** Đảm nhận việc chấm dứt kết nối SSL/TLS (SSL Termination), phân phối lưu lượng tải (Round-robin/Least Connections) tới các Pod Spring Boot.
- **Auto-Scaling Strategy:** Cấu hình Kubernetes Horizontal Pod Autoscaler (HPA) giám sát chỉ số CPU/RAM. Tự động scale từ 2 Pod lên tối đa 10 Pod khi lượng CPU trung bình của cluster vượt quá 70%.

- **Database Isolation:** Database được đưa vào Private Subnet không thể truy cập trực tiếp từ Internet, chỉ cho phép kết nối nhận từ IP Range của App Subnet để bảo vệ an toàn dữ liệu.

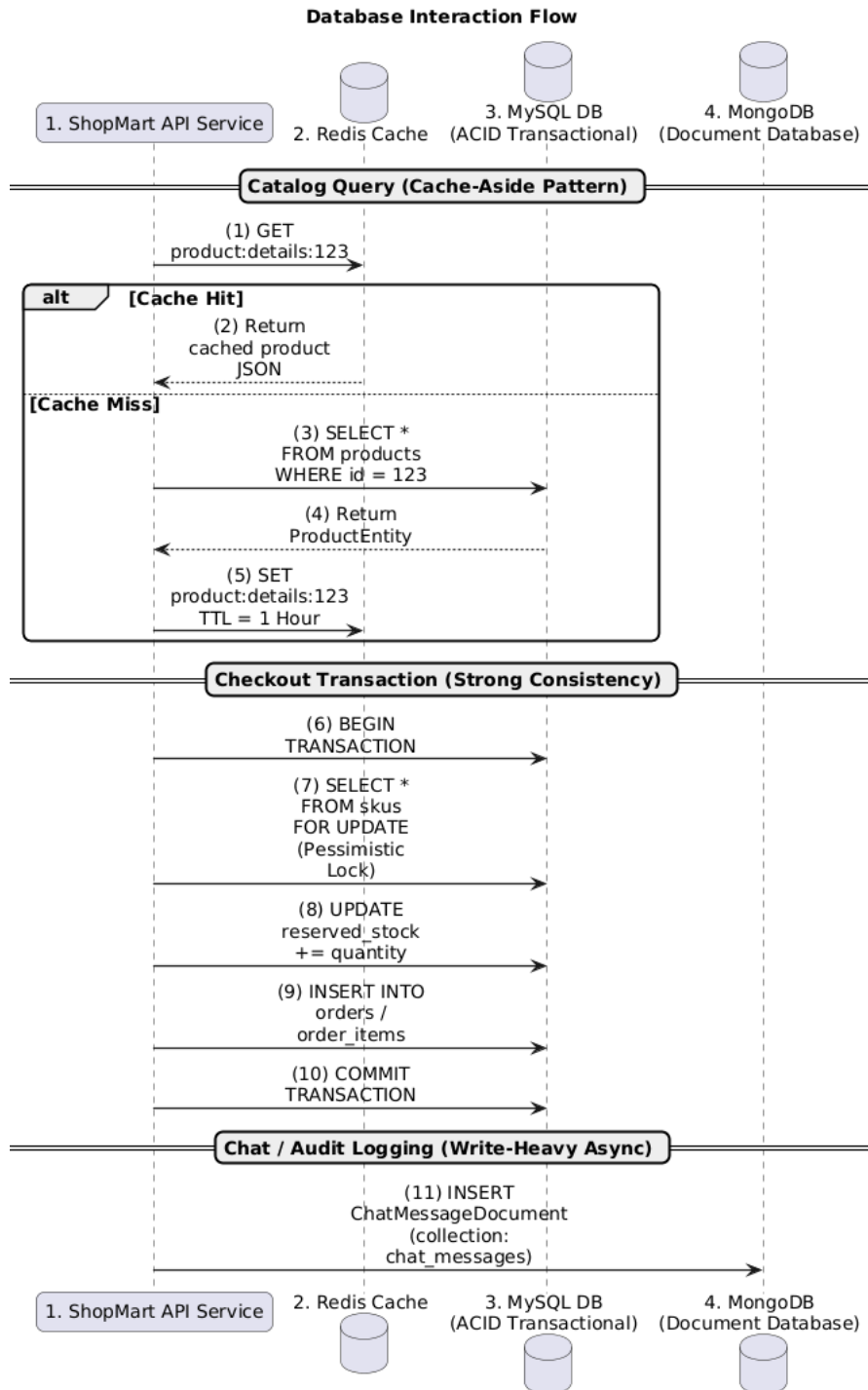
4 Data architecture

Kiến trúc dữ liệu của ShopMart được thiết kế để đảm bảo sự cân bằng giữa tính nhất quán cao (ACID) của giao dịch thương mại và tốc độ xử lý nhanh (throughput) của các luồng đọc/ghi phi cấu trúc.

4.1 Hybrid Storage Strategy

- MySQL Database (RDBMS): Lưu trữ thực thể nghiệp vụ có cấu trúc chặt chẽ. Hệ thống sử dụng Flyway để tự động hóa các phiên bản schema (V0001 đến V0139), đảm bảo tính đồng bộ cấu trúc bảng trên mọi môi trường (Dev, Staging, Production).
- MongoDB (NoSQL Document Store): Lưu trữ tài liệu. Thiết kế Collection:
 - chat_messages: Lưu tin nhắn chat thời gian thực.
 - event_log: Nhật ký sự kiện bảo mật (Audit log) của Admin.
 - user_event & user_category_affinity: Phân tích hành vi (click, xem sản phẩm) phục vụ hệ thống gợi ý (Recommendation System).
- Redis Cache: Lưu trữ dạng Key-Value trong RAM. Áp dụng cơ chế Write-through hoặc Cache-aside để cache các thông tin danh mục (categories), thương hiệu (brands), thông số so sánh và sản phẩm xem nhiều.

4.2 Database Interaction Diagram



4.3 Caching & Locking Mechanisms

Pessimistic Write Lock (SELECT ... FOR UPDATE): Được khai báo tại `SkuJpaRepository` bằng annotation của Spring Data JPA:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT s FROM SkuEntity s WHERE s.id = :id")
Optional<SkuEntity> findByIdForUpdate(@Param("id") Long id);
```

- Khi một thread gọi hàm này, Hibernate sẽ sinh câu lệnh SQL `SELECT ... FOR UPDATE` khóa dòng dữ liệu của SKU đó trong MySQL. Các thread khác cố gắng đọc dòng này để checkout cùng lúc sẽ phải xếp hàng chờ cho đến khi thread đầu tiên hoàn tất transaction (commit hoặc rollback). Điều này đảm bảo tuyệt đối không bao giờ bị overselling.
- **Token Blacklisting trong Redis:** Khi người dùng gọi API /logout, Access Token hiện tại (vẫn còn hạn) được đưa vào Redis blacklist với key `auth:blacklist:{jti}` và TTL bằng thời gian sống còn lại của token. Mỗi request đi qua `JwtAuthFilter` đều được kiểm tra nhanh trong Redis; nếu token nằm trong blacklist, hệ thống lập tức trả về lỗi 401 Unauthorized.

5 Api architecture

ShopMart Backend cung cấp hệ thống RESTful API chuẩn hóa, phiên bản hóa thông qua tiền tố `/api/v1/`.

5.1 REST Conventions & Error Handling

API Response Wrapper: Tất cả các API đều trả về một định dạng JSON thống nhất sử dụng class `ApiResponse<T>`:

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... }
}
```

- **Global Exception Handler:** Hệ thống sử dụng `@RestControllerAdvice` (`CustomErrorController.java`) để bắt mọi Exception xảy ra trong runtime và map thành mã lỗi HTTP tương ứng:
 - `ApiException.notFound(...)` 404 Not Found
 - `ApiException.insufficientStock(...)` 409 Conflict
 - `MethodArgumentNotValidException` 400 Bad Request (trả về chi tiết các trường validate lỗi).

5.2 API Catalog

Endpoint	Method	Auth Role	Module	Request DTO	Response DTO	Description
/api/v1/auth/register	POST	permit All()	Auth	Register Request	ApiResponse<TokenResponse>	Đăng ký tài khoản khách hàng mới.
/api/v1/auth/login	POST	permit All()	Auth	LoginRequest	ApiResponse<TokenResponse>	Đăng nhập bằng Email/Password, trả về JWT Token Pair.
/api/v1/auth/otp/send	POST	permit All()	Auth	SendOtpRequest	ApiResponse<MessageResponse>	Gửi mã OTP đăng nhập đến

						số điện thoại.
/api/v1/auth/otp/verify	POST	permit All()	Auth	VerifyOTPRequest	ApiResponse<TokenResponse>	Xác minh OTP và đăng nhập/tạo tài khoản nhanh.
/api/v1/auth/trusted-login	POST	permit All()	Auth	TrustedLoginRequest	ApiResponse<TokenResponse>	Đăng nhập bỏ qua OTP bằng Device ID tin cậy.
/api/v1/auth/register-seller	POST	Authenticated	Auth	None	ApiResponse<TokenResponse>	Nâng cấp tài khoản hiện tại lên vai trò người bán (Seller).
/api/v1/cart/items	POST	ROLE_CLIENT	Cart	AddToCartRequest	ApiResponse<Long>	Thêm một biến thể sản phẩm (SKU) vào giỏ hàng.

/api/v1/checkout	POST	ROLE_CLIENT	Checkout	CheckoutRequest	ApiResponse<List<OrderResponse>>	Đặt hàng đa cửa hàng. Yêu cầu Idempotency-Key ở Header.
/api/v1/payment/momo/callback	POST	permit All()	Payment	Request Params	MessageResponse	Webhook nhận kết quả thanh toán từ MoMo (IPN).
/api/v1/payment/vnpay/callback	GET	permit All()	Payment	Request Params	Redirect String	Callback nhận kết quả thanh toán VNPAY để redirect client.
/api/v1/seller/inventory/logs	GET	ROLE_SELLER	Inventory	Query Params	ApiResponse<Page<InventoryLog>>	Xem nhật ký biến động xuất/nhập kho của shop.

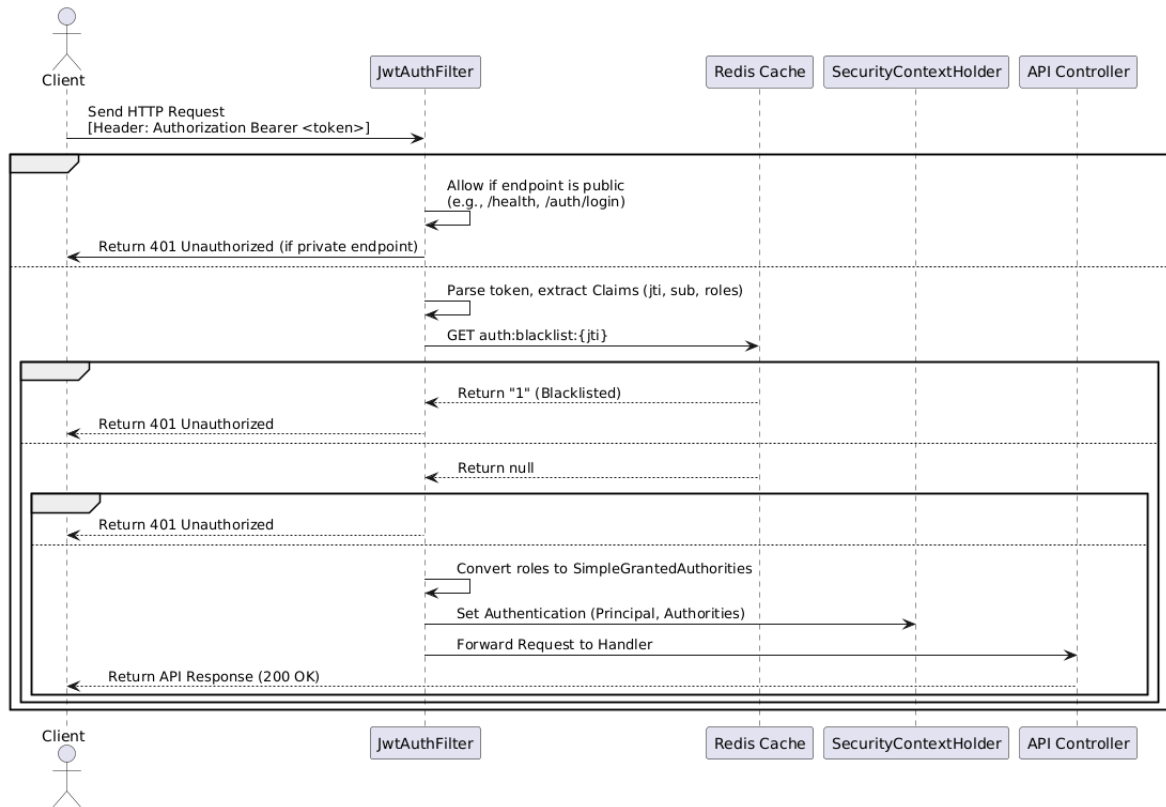
/api/v1/compare	GET	permit All()	Catalog	Query Params	ApiResponse<CompareResponse>	So sánh thông số kỹ thuật chi tiết của các sản phẩm.
/api/v1/ai-assistant/query	POST	permit All()	AI	AiAssistantRequest	ApiResponse<AiAssistantResponse>	Gửi câu hỏi tư vấn sản phẩm tới Chatbot AI.
/api/v1/admin/audit-logs	GET	ROLE_ADMIN	Audit	Query Params	ApiResponse<Page<EventLog>>	Xem lịch sử thao tác hệ thống của Admin.

6 Security architecture

Kiến trúc bảo mật của ShopMart được thiết kế để đảm bảo an toàn tối đa cho các giao dịch tài chính và ngăn chặn các hành vi tấn công phổ biến.

6.1 Authentication Sequence Diagram

Chi tiết quy trình xác thực JWT, kiểm tra phiên làm việc và lọc yêu cầu tại API Layer:



6.2 Role-Based Access Control (RBAC) Matrix

API Path Prefix	HTTP Method	Public	Buyer	Seller	Admin
/api/v1/auth/register, /login	POST	✓	✓	✓	✓
/api/v1/catalog/public/**	GET	✓	✓	✓	✓
/api/v1/cart/**	ALL	✗	✓	✗	✗
/api/v1/checkout/**	POST	✗	✓	✗	✗
/api/v1/seller/profile/**	ALL	✗	✗	✓	✗
/api/v1/seller/catalog/**	ALL	✗	✗	✓	✗
/api/v1/seller/analytics/**	GET	✗	✗	✓	✗
/api/v1/admin/**	ALL	✗	✗	✗	✓

6.3 Security Boundary & Mitigations

- CSRF (Cross-Site Request Forgery): Bị vô hiệu hóa (`http.csrf(csrf -> csrf.disable())`) tại `SecurityConfig.java`. Vì hệ thống sử dụng stateless JWT authentication được đính kèm ở Header (`Authorization: Bearer ...`) thay vì lưu session trong Cookie của trình duyệt, ứng dụng tự động miễn nhiễm với các cuộc tấn công CSRF.
- CORS (Cross-Origin Resource Sharing): Được cấu hình chặt chẽ qua `CorsProperties` và `corsConfigurationSource()`. Chỉ cho phép các domain frontend được cấu hình cụ thể trong biến môi trường `APP_CORS_ALLOWED_ORIGINS` (ví dụ: `http://localhost:3000`, `https://shopmart.com`) truy cập API, chặn hoàn toàn các request cross-origin từ các domain lạ.
- Mã hóa dữ liệu nhạy cảm: Mật khẩu của người dùng được hash bằng BCrypt trước khi lưu xuống cơ sở dữ liệu. Khóa bí mật JWT (`JWT_SECRET_BASE64`) được mã hóa Base64 và chỉ nạp qua cấu hình môi trường bảo mật, không commit trực tiếp lên git.

7 Quality attribute analysis

7.1 Performance

- Tactic áp dụng:
 - Caching: Cấu hình Spring Cache kết hợp Redis (`CacheConfig.java`) để lưu trữ kết quả của các query nặng (như danh sách danh mục sản phẩm, cấu hình specs so sánh).
 - Database Connection Pooling: Sử dụng HikariCP làm connection pool mặc định với kích thước được tối ưu hóa để giảm chi phí thiết lập kết nối tới MySQL.
 - Batch Fetching: Sử dụng `findAllById` để gom truy vấn (như load danh sách SKU và Product ở OrderService dòng 94-110) thay vì chạy câu lệnh SELECT trong vòng lặp.

- Evidence in Code: `@Cacheable(value = "categories", key = "'all'")` trong `CatalogService.java`.
- Bottlenecks & Risks: Redis Single Instance có thể trở thành điểm nghẽn cổ chai nếu lượng traffic đọc quá lớn.
- Khuyến nghị: Nâng cấp hạ tầng Redis lên Redis Sentinel hoặc Redis Cluster để tăng throughput đọc qua các replica nodes.

7.2 Scalability

- Tactic áp dụng:
 - Stateless Service Design: Toàn bộ API controller và Application service hoàn toàn không lưu trạng thái (no local HTTP session). Mọi thông tin phiên làm việc được chứa trong token JWT hoặc Redis. Điều này cho phép mở rộng hệ thống theo chiều ngang (Horizontal Scaling) bằng cách nhân bản thêm Pod Spring Boot mà không cần cấu hình đồng bộ session phức tạp.
 - Evidence in Code: `SessionCreationPolicy.STATELESS` được thiết lập tại `SecurityConfig.java`.
 - Bottlenecks & Risks: Database MySQL ghi (Primary Node) là điểm nghẽn vật lý duy nhất khi lượng transaction mua hàng tăng đột biến.
- Khuyến nghị: Áp dụng Read-Write Splitting (phân tách kết nối Đọc/Ghi). Đẩy toàn bộ các truy vấn tìm kiếm, xem sản phẩm, xem lịch sử đơn sang MySQL Replica Nodes, dành riêng MySQL Primary Node cho ghi checkout/payment.

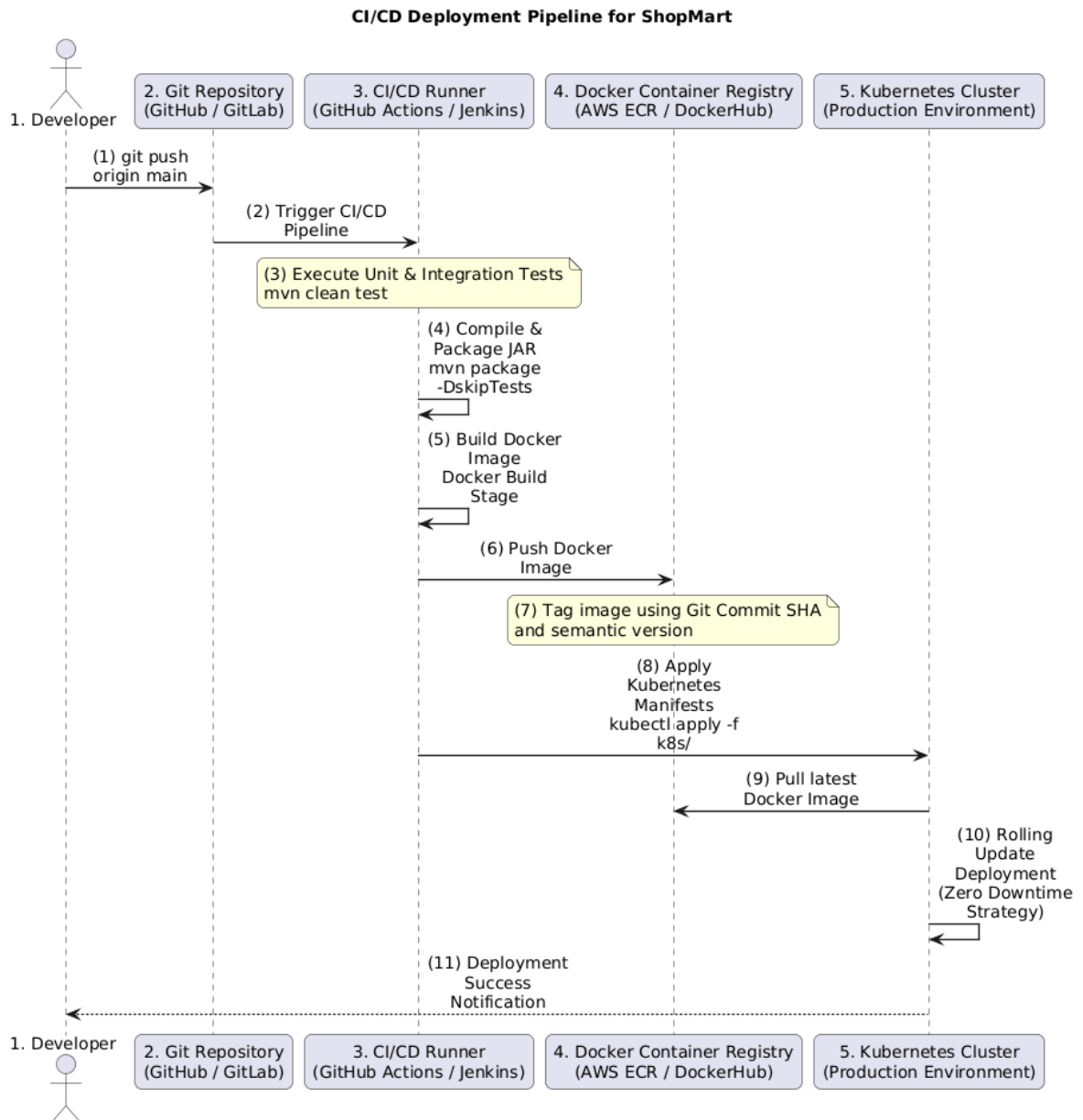
7.3 Observability & Auditability

- Tactic áp dụng:
 - Structured JSON Logging: Tích hợp Logback với encoder Logstash (logstash-logback-encoder trong pom.xml) để ghi log dạng JSON có cấu trúc.

- Metrics Exposure: Sử dụng Micrometer kết hợp Spring Boot Actuator để thu thập và expose các metrics của JVM, HTTP request, và Hibernate query sang định dạng Prometheus.
- Audit Event Log: Tự động lưu vết hành vi Admin vào MongoDB.
- Evidence in Code: `management.endpoints.web.exposure.include: health,info,metrics,prometheus` tại `application.yaml`.
- Bottlenecks & Risks: File log phình to nhanh chóng làm cạn kiệt dung lượng đĩa cứng của server.
- Khuyến nghị: Thiết lập cơ chế xoay vòng log (Log Rotation) và đẩy log tập trung về hệ thống Elasticsearch/Logstash/Kibana (ELK Stack) hoặc Grafana Loki.

8 Devops & operations

8.1 DevOps Pipeline Diagram



8.2 Logging, Monitoring & Alerting Stack

Metrics: Prometheus định kỳ cào (scrape) dữ liệu từ endpoint `/actuator/prometheus` của ứng dụng Spring Boot để lấy thông tin về: số lượng request lỗi (HTTP 5xx), thời gian phản hồi trung bình, dung lượng Heap Memory sử dụng.

Visualizing: Grafana kết nối với Prometheus để vẽ biểu đồ dashboard giám sát thời gian thực.

Alerting: Grafana Alerting hoặc Prometheus Alertmanager cấu hình gửi thông báo khẩn cấp qua Telegram/Slack cho đội ngũ SRE khi:

Tỷ lệ request lỗi 5xx vượt quá 5% trong vòng 2 phút.

Dung lượng bộ nhớ RAM của Pod vượt quá 90%.

Database connection pool bị cạn kiệt (Active Connections = Max Connections).

9 Risks & technical debt

Risk / Debt	Severity	Evidence in Codebase	Impact	Recommendation
Bảo mật mật khẩu lỗi (Plaintext Fallback)	High	PasswordHasher.matches dòng 15 so sánh trực tiếp chuỗi thô: hashed.equals(raw).	Nếu hacker truy cập được database, họ có thể đăng nhập trực tiếp bằng các tài khoản có mật khẩu chưa hash (plaintext).	Loại bỏ hoàn toàn dòng so sánh plaintext này. Viết script migration băm lại toàn bộ dữ liệu mật khẩu cũ.
Thiếu kiểm tra quyền sở hữu đối tượng (IDOR)	Medium	Các controller như OrderController kiểm tra sở hữu thủ công bằng code Java thay vì cấu hình khai báo.	Dễ bỏ sót kiểm tra ở các API mới phát triển, dẫn đến lỗi hỏng xem/sửa trái phép tài nguyên của user khác.	Áp dụng @PreAuthorize kết hợp custom security expressions ở mức method để chuẩn hóa việc check quyền sở hữu.
Không có cơ chế bù trừ dữ liệu (Saga/Outbox)	Medium	AuthService.java dòng 107-116 bọc lệnh lưu MongoDB	Nếu MongoDB gặp sự cố, hệ thống vẫn cho phép user đăng	Triển khai Transactional Outbox Pattern : Lưu sự kiện log vào bảng

box) cho MongoDB logging		trong try-catch trống.	nhập nhưng Audit Log bị mất, gây lệch dữ liệu giám sát.	MySQL trước, sử dụng Worker quét đầy sang Mongo bất đồng bộ.
Thiếu Rate Limiting cho API gửi OTP/Logi n	High	Đã khai báo thư viện bucket4j nhưng chưa áp dụng Interceptor/Filter cho endpoint /api/v1/auth/otp/send.	Hệ thống dễ bị tấn công spam OTP gây thiệt hại chi phí viễn thông, hoặc brute-force đoán OTP/mật khẩu.	Viết một Spring HandlerInterceptor sử dụng RateLimitService để giới hạn số lần gọi API gửi OTP từ một địa chỉ IP.
Nguy cơ lỗi N+1 Query khi load Order List	Medium	Gọi productRepo.findAllById lặp đi lặp lại trong luồng duyệt map đơn hàng (OrderService.java dòng 288-290).	Hiệu năng truy vấn SQL giảm mạnh khi danh sách đơn hàng trả về lớn.	Sử dụng JPA @EntityGraph hoặc JOIN FETCH để load toàn bộ dữ liệu liên quan trong một câu lệnh SELECT duy nhất.

10 Architecture decision records (adr)

ADR ID	Decision	Rationale	Alternatives Considered	Consequences
ADR-01	Áp dụng kiến trúc Clean Architecture	Tách biệt nghiệp vụ lõi khỏi framework và cơ sở dữ liệu để	Traditional 3-Tier Layered Architecture	Tăng số lượng class trung gian và code mapping. Yêu cầu lập trình viên phải

		để bảo trì, nâng cấp và viết unit test.		hiểu rõ quy tắc ranh giới layer.
ADR-02	Sử dụng Pessimistic Locking cho luồng checkout kho	Ngăn chặn tuyệt đối tình trạng overselling khi lượng đặt hàng đồng thời cực cao.	1. Optimistic Locking 2. In-memory locking (Redis Redlock)	Có thể làm tăng nhẹ thời gian phản hồi của transaction checkout do các thread phải xếp hàng chờ lock ở Database.
ADR-03	Lưu dữ liệu Chat và Log sang MongoDB	Giảm tải cho MySQL, tối ưu hóa lưu trữ dữ liệu tài liệu phi cấu trúc và tăng khả năng ghi chép hiệu năng cao.	Lưu toàn bộ trong MySQL	Hệ thống tăng thêm phụ thuộc hạ tầng (cần quản lý thêm cluster MongoDB). Phát sinh vấn đề nhất quán dữ liệu lai (Hybrid Consistency).
ADR-04	Bảo mật Webhook bằng Signature HMAC-SHA512	Chống giả mạo kết quả thanh toán từ hacker gửi trực tiếp tới callback API.	Kiểm tra IP Whitelisting của cổng thanh toán	Đảm bảo an toàn tuyệt đối. Cần quản lý an toàn khóa bí mật (hash-secret) trên môi trường production.
ADR-05	Idempotent Checkout qua Idempotency-Key	Phòng ngừa tạo trùng đơn hàng do Buyer bấm nút Đặt hàng nhiều lần hoặc	Kiểm tra trùng lặp bản ghi thô ở Database	Yêu cầu Client phải tạo và quản lý UUID gửi kèm ở header. Backend cần hạ tầng Redis hoạt động ổn

		do cơ chế retry tự động của mạng.		định để lưu trạng thái lock key.
--	--	-----------------------------------	--	----------------------------------

11 Requirement traceability

Requirement / Use Case	Backend Module	API Endpoint	Processing Component	Database Tables
Đăng ký tài khoản Buyer	Authentication	/api/v1/auth/register	AuthController AuthService	users, user_profiles, user_roles
Đăng nhập OTP & Đăng ký thiết bị	Authentication	/api/v1/auth/otp/verify	AuthController AuthService	users, trusted_devices, refresh_sessions
Đăng ký Shop	Seller Shop	/api/v1/auth/register-seller	AuthController AuthService	seller_profiles, seller_shops, user_roles
Thêm sản phẩm vào giỏ hàng	Cart	/api/v1/cart/items	CartController CartService	cart_items
Giữ chỗ tồn kho tạm thời	Inventory	/api/v1/checkout	OrderService ReservationService	skus, stock_reservations

Đặt hàng đa cửa hàng	Checkout	/api/v1/checkout	CheckoutController OrderService	orders, order_items
Callback thanh toán an toàn	Payment	/api/v1/payments/momo/callback	PaymentController PaymentService	payments, orders
Chat Buyer-Seller thời gian thực	Chat / Support	/api/v1/chat/send	ChatController ChatService	MongoDB collection: chat_messages
Tư vấn mua hàng qua AI	AI Assistant	/api/v1/ai-assistant/query	AiAssistantController AiAssistantService	Read from products
Xem nhật ký hệ thống của Admin	Audit Log	/api/v1/admin/audit-logs	AdminAuditController AuditService	MongoDB collection: event_log